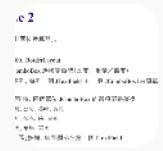
物件導向程式設計II

姓名:李欣曈 學號:A1143302

HW3-10/09-AI 對話紀錄



請根據作業要求，一步一步教我用Java Swing寫一個『單位換算器』，並給出完整程式碼。

根據您提供的作業規格，我們可以使用 Java Swing 來一步步實作這個「單位換算器」。為了解釋清楚，我們將按照版面配置的五個區域來拆解： PNG

實作步驟解說：

1. **第一步：設定主視窗與版面配置 (JFrame)**
   題目要求視窗尺寸為 `480x280`，並使用 `BorderLayout`（邊界版面配置）。這種配置可以將視窗完美切分為上(NORTH)、中(CENTER)、下(SOUTH)。 PNG
2. **第二步：配置上方區域 (NORTH)**
   建立一個 `JComboBox` 下拉選單，放入「長度、重量、溫度」三個選項，並放置在視窗的 NORTH 位置。 PNG
3. **第三步：配置中間區域 (CENTER)**
   題目要求兩列，每列包含一個輸入框 (`JTextField`) 和一個單位選單 (`JComboBox`)。我們可以用一個 `JPanel` 並設定為 `GridL t(2, 2)` （兩列兩行）來包裝這四個元件，再整個放入 CENTER。 PNG +1

3. **第三步：配置中間區域 (CENTER)**
   題目要求兩列，每列包含一個輸入框（`JTextField`）和一個單位選單（`JComboBox`）。我們可以用一個 `JPanel` 並設定為 `GridLayout(2, 2)`（兩列兩行）來包裝這四個元件，再整個放入 CENTER。 `PNG+1`
4. **第四步：配置下方區域 (SOUTH)**
   建立一個「換算」按鈕（`JButton`），並將它放置在視窗的 SOUTH 位置。 `PNG`
5. **第五步：加入事件處理 (連動與計算)**
   - **連動事件**：當切換 NORTH 的類型選單時，動態替換 CENTER 兩個單位選單的內容。 `PNG`
   - **計算事件**：當按下換算按鈕時，讀取第一個 `JTextField` 的數字，根據選取的單位進行數學換算，並將結果顯示在第二個 `JTextField`。 `PNG`

完整程式碼：

這是一份符合您簡報風格（使用靜態變數、內部類別傾聽者）的完整程式碼，你可以將它命名為 `HW3.java` 執行看看：

Java

```java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class HW3 {
    // --- 1. 宣告全域靜態 UI 元件 ---
    static JFrame frm = new JFrame("單位換算器");
    static JPanel centerPanel = new JPanel();

    // 宣告下拉選單元件
    static JComboBox typeCombo;
    static JComboBox sourceCombo;
    static JComboBox targetCombo;

    static JTextField inputField = new JTextField();
    static JTextField resultField = new JTextField();
    static JButton calcButton = new JButton("換算");

    // --- 宣告單位陣列選項[cite: 72] ---
    static String[] types = {"長度", "重量", "溫度"};
    static String[] lengthUnits = {"公尺", "公分", "英吋", "英尺"};
    static String[] weightUnits = {"公斤", "公克", "磅", "盎司"};
    static String[] tempUnits = {"攝氏", "華氏", "克氏"};

    public static void main(String args[]) {
        // --- 設定視窗與排版[cite: 72] ---
        frm.setSize(480, 280);
        frm.setLayout(new BorderLayout());

        // --- 配置上方區域 (NORTH)[cite: 72] ---
        typeCombo = new JComboBox(types);
        frm.add(typeCombo, BorderLayout
```

Java

```
        frm.add(typeCombo, BorderLayout.NORTH);

        // --- 配置中間區域 (CENTER)[cite: 72] ---
        centerPanel.setLayout(new GridLayout(2, 2, 5, 5)); // 兩列兩行
        sourceCombo = new JComboBox(lengthUnits);
        targetCombo = new JComboBox(lengthUnits);
        resultField.setEditable(false); // 結果框設為唯讀

        // 依序加入：左上、右上、左下、右下
        centerPanel.add(inputField);
        centerPanel.add(sourceCombo);
        centerPanel.add(resultField);
        centerPanel.add(targetCombo);
        frm.add(centerPanel, BorderLayout.CENTER);

        // --- 配置下方區域 (SOUTH)[cite: 72] ---
        frm.add(calcButton, BorderLayout.SOUTH);

        // --- 註冊事件傾聽者 ---
        typeCombo.addActionListener(new ActLis());
        calcButton.addActionListener(new ActLis());

        // 顯示視窗
        frm.setVisible(true);
        frm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    // --- 內部類別處理事件 ---
    static class ActLis implements ActionListener {
        public void actionPerformed(ActionEvent e) {

            // 事件一：切換換算類型時，連動更換單位選單[cite: 72]
            if (e.getSource() == typeCombo) {
                int selectedIndex = typ      bo.getSelectedIndex();
                String[] newUnits = leng     nits; // 預設長度
```

Java

```
                int selectedIndex = typeCombo.getSelectedIndex();
                String[] newUnits = lengthUnits; // 預設長度

                if (selectedIndex == 1) newUnits = weightUnits; // 重量
                else if (selectedIndex == 2) newUnits = tempUnits; // 溫度

                // 重新設定選單模型，並清空輸入框
                sourceCombo.setModel(new DefaultComboBoxModel(newUnits));
                targetCombo.setModel(new DefaultComboBoxModel(newUnits));
                resultField.setText("");
                inputField.setText("");
            }

            // 事件二：按下換算按鈕，執行計算顯示於第二個 JTextField[cite: 72]
            else if (e.getSource() == calcButton) {
                try {
                    double inputValue = Double.parseDouble(inputField.getText());
                    int typeIdx = typeCombo.getSelectedIndex();
                    int srcIdx = sourceCombo.getSelectedIndex();
                    int tgtIdx = targetCombo.getSelectedIndex();
                    double result = 0.0;

                    // 長度換算邏輯
                    if (typeIdx == 0) {
                        double[] toMeter = {1.0, 0.01, 0.0254, 0.3048};
                        result = (inputValue * toMeter[srcIdx]) / toMeter[tgtIdx];
                    }
                    // 重量換算邏輯
                    else if (typeIdx == 1) {
                        double[] toKg = {1.0, 0.001, 0.453592, 0.0283495};
                        result = (inputValue * toKg[srcIdx]) / toKg[tgtIdx];
                    }
                    // 溫度換算邏輯
                    else if (typeIdx ==    {
                        double inCelsiu    inputValue;
                        if (srcIdx == 1) inCelsius = (inputValue - 32) * 5.0 / 9.0;
```

```java
                    double[] toKg = {1.0, 0.001, 0.453592, 0.0283495};
                    result = (inputValue * toKg[srcIdx]) / toKg[tgtIdx];
                }
                // 溫度換算邏輯
                else if (typeIdx == 2) {
                    double inCelsius = inputValue;
                    if (srcIdx == 1) inCelsius = (inputValue - 32) * 5.0 / 9.0;
                    else if (srcIdx == 2) inCelsius = inputValue - 273.15;

                    if (tgtIdx == 0) result = inCelsius;
                    else if (tgtIdx == 1) result = inCelsius * 9.0 / 5.0 + 32;
                    else if (tgtIdx == 2) result = inCelsius + 273.15;
                }

                // 顯示結果到小數點後 4 位
                resultField.setText(String.format("%.4f", result));
            } catch (Exception ex) {
                resultField.setText("錯誤");
            }
        }
    }
}
```

這段程式碼完全符合題目的 UI 排版需求與互動邏輯。你可以先在編輯器貼上執行看看，如果遇到任何問題我們再來一步步解決！

為什麼執行出來後程式可以跑，但有JComboBox的地方都會出現黃色底線警告，請問這是什麼原因？該怎麼解決？

這些黃色底線是編輯器發出的**警告（Warnings）**，而不是錯誤（Errors），所以程式依然可以正常編譯和執行。

這是什麼原因？
這是因為現代的 Java 規範中，像 `JComboBox` 這種用來裝資料的「容器」，官方強烈建議要使用 `< >` **(泛型 Generics)** 來標明裡面裝的是什麼資料（例如文字 `<String>`）。
剛才的程式碼為了配合簡報中較早期的語法示範，沒有加上 `< >`，所以編輯器會跳出「缺乏型態安全檢查（Raw use of parameterized class）」的警告，提醒你這是不標準的寫法。

該怎麼解決？
要消除這些黃線，你只需要把 `<String>` 的規定補回去。請修改程式碼中的這三個地方：

1. 修改宣告變數的地方：告訴程式選單專門用來裝文字。
將原本的：

```java
static JComboBox typeCombo;
static JComboBox sourceCombo;
static JComboBox targetCombo;
```

改成：

改成：

```java
static JComboBox<String> typeCombo;
static JComboBox<String> sourceCombo;
static JComboBox<String> targetCombo;
```

2. **修改建立物件的地方：在 new JComboBox 後面加上 `<>`。**
將原本的：

```java
typeCombo = new JComboBox(types);
sourceCombo = new JComboBox(lengthUnits);
targetCombo = new JComboBox(lengthUnits);
```

改成：

```java
typeCombo = new JComboBox<>(types);
sourceCombo = new JComboBox<>(lengthUni
targetCombo = new JComboBox<>(lengthUni
```

3. **修改更換選單內容的地方：DefaultComboBoxModel 也要加上 `<>`。**
將原本的：

```java
sourceCombo.setModel(new DefaultComboBoxModel(newUnits));
targetCombo.setModel(new DefaultComboBoxModel(newUnits));
```

改成：

```java
sourceCombo.setModel(new DefaultComboBoxModel<>(newUnits));
targetCombo.setModel(new DefaultComboBoxModel<>(newUnits));
```

只要把這三個地方的 `<String>` 和 `<>` 補齊，存檔後黃色底線就會全部消失，程式也會變得更安全、更符合現代 Java 的標準規範！

HW3-10/09- Reflection

1. AI 給的哪裡有問題：

   剛開始 AI 提供的程式碼在宣告 JComboBox 與 DefaultComboBoxModel 時，沒有使用泛型標籤，例如直接寫成 static JComboBox typeCombo;。雖然程式可以順利編譯與執行，但編譯器會出現黃色底線警告。

2. 改了什麼：

   我把程式碼中所有和 JComboBox 跟 DefaultComboBoxModel 相關的宣告與建立物件的地方，都補上了<String>或<>。具體修改了三個部分：

   - **宣告變數：** 將 static JComboBox typeCombo; static JComboBox sourceCombo; static JComboBox targetCombo; 改為 static JComboBox<String> typeCombo; static JComboBox<String> sourceCombo; static JComboBox<String> targetCombo;。
   - **建立物件：** 將 typeCombo = new JComboBox(types); sourceCombo = new JComboBox(lengthUnits); targetCombo = new JComboBox(lengthUnits);改為 typeCombo = new JComboBox<>(types); sourceCombo = new JComboBox<>(lengthUnits); targetCombo = new JComboBox<>(lengthUnits);。

- **設定選單模型：** 將 sourceCombo.setModel(new DefaultComboBoxModel(newUnits)); targetCombo.setModel(new DefaultComboBoxModel(newUnits));改為 sourceCombo.setModel(new DefaultComboBoxModel<>(newUnits)); targetCombo.setModel(new DefaultComboBoxModel<>(newUnits));。

3. 為什麼改：

   因為補上<String>可以明確告訴 Java 編譯器，這些下拉選單只能裝文字資料，這樣做可以成功消除黃色底線警告。